

In the name of ALLAH

به امید نابودی رژیم نجس کودک کش



Amirkabir University of Technology
(Tehran Polytechnic)

Deep Neural Network Course By Dr. Safabakhsh

spring 2025

Assignment (06.1)

Mohammad Sadegh Eftekhari

403131014

Sadegh.eft@aut.ac.ir

Contents

IMPORTS	3
Before we begin	4
Q1: Data Preparation	4
Q2: DCGAN Architecture	4
Generator	4
Discriminator	5
Weight Initialization	5
The DCGAN Training Loop	5
Update the Discriminator Network	5
Update the Generator Network	6
Evaluating GAN Performance: FID and IS	6
Q3: Hyperparameter Sensitivity Analysis	7
1. Input Noise Vector Size (z_{dim})	7
2. Batch Normalization	7
3. Label Smoothing	7
4. Activation Functions	8
Results (Numerical) - 5 epochs	8
Results (Visual) - 5 epochs	10
Q4: Conditional GAN (CGAN)	16
Architectural Modifications	16
Generator (Generator_CGAN):	16
Discriminator (Discriminator_CGAN):	16
Training and Conditional Generation	17
Results (All Classes)	17
Results (Specific Classes)	18
Q5: Wasserstein GAN with Gradient Penalty (WGAN-GP)	18
Architectural and Methodological Changes	19
Training Process	19
Results (epoch 1 – 5)	20
Q6: Distribution Visualization with t-SNE	21
Methodology	21
Interpreting the Results	21

Q7: Analysis of GANs.....	22
GANs vs. Other Generative Models (like VAEs)	22
Core Training Challenges.....	22
Solutions for Stable Training.....	22

IMPORTS

1. `torch/torch.nn/torch.nn.functional`:
The core deep learning framework used for building the Generator and Discriminator models, defining loss functions, and managing tensor operations.
2. `torchvision`:
Provides easy access to standard datasets like CIFAR-10 and includes helpful transformations (transforms) and utilities (save_image).
3. `torch.utils.data.DataLoader`:
Essential for creating efficient data pipelines, allowing us to load and batch our image data for training.
4. `torch_fidelity`:
A crucial library for evaluation, used to calculate standard GAN metrics like Fréchet Inception Distance (FID) and Inception Score (IS).
5. `os/timeit.default_timer`:
Used for system-level operations like creating directories for saving images and for timing our training loops.
6. `matplotlib.pyplot/seaborn/pandas`:
Our toolkit for data visualization. Matplotlib is used for plotting generated images and loss curves, while Pandas and Seaborn are used together to create structured t-SNE plots for visualizing data distributions.
7. `tqdm.auto`:
Provides progress bars to monitor the training progress of our models, which is especially helpful during long-running epochs.
8. `sklearn.manifold.TSNE`:
A key algorithm from Scikit-learn used for dimensionality reduction to visualize the high-dimensional feature space of our models in 2D.

Before we begin

Before getting into the main parts of the code, we have to set some things up.

The 1st thing is to filter any warnings right out the gate, we do this by calling the method from the 'warnings' library and passing the value of 'ignore'. We then want to setup our device, if the 'CUDA' is available we use that, if not we just use our 'CPU'. The last thing is to setup a timer function which is going to calculate the time and print it for us.

Q1: Data Preparation

Our data preparation pipeline is straightforward:

1. **Transformations:** We define a transform pipeline using "torchvision.transforms". Each image is first converted into a PyTorch tensor. Then, we apply a normalization step. Specifically, we normalize the image tensors to have a mean of 0.5 and a standard deviation of 0.5. This shifts the pixel values from the [0, 1] range to the [-1, 1] range. This is a critical step, as it makes the input data compatible with the Generator's final "nn.Tanh" activation function, which also outputs values in the [-1, 1] range.
2. **DataLoaders:** We create a "DataLoader" for the training set. This loader shuffles the data and groups it into batches. Following common practice for DCGANs, we use a batch size of 128.

Q2: DCGAN Architecture

At the heart of our project is the Deep Convolutional Generative Adversarial Network (DCGAN). This architecture uses a pair of dueling neural networks: a Generator that creates images and a Discriminator that critiques them.

Generator

The Generator's job is to create realistic-looking images from random noise. Our implementation follows the core DCGAN principles:

- **Input:** It takes a random noise vector z (from a 100-dimensional latent space) as input.
- **Architecture:** It's built using a series of nn.ConvTranspose2d layers. These layers perform transposed convolutions (often called deconvolutions) to progressively "upsample" the low-resolution noise vector into a full-sized 32x32x3 image.
- **Stabilization:** Between the convolutional layers, we use nn.BatchNorm2d to stabilize training and "nn.ReLU" as the activation function to introduce non-linearity.
- **Output Layer:** The final layer uses a "nn.Tanh" activation function. As mentioned earlier, this ensures the generated image's pixel values are in the [-1, 1] range, matching our normalized real images.

Discriminator

The Discriminator acts as a binary classifier, trying to determine if an image is real (from the dataset) or fake (from the Generator).

- **Input:** It takes a 32x32x3 image.
- **Architecture:** It's a standard convolutional neural network (CNN) composed of `nn.Conv2d` layers. These layers “downsample” the image, extracting features at each step.
- **Activation Function:** We use “`nn.LeakyReLU`” with a slope of 0.2. This is a key feature of DCGANs, as it helps prevent the “dying ReLU” problem and allows gradients to flow more easily, which is crucial for the Generator to learn effectively.
- **Output Layer:** The network ends with a single `nn.Conv2d` layer that flattens the features into a single logit, followed by an “`nn.Sigmoid`” activation. This squashes the output to a probability between 0 (fake) and 1 (real).

Weight Initialization

Proper weight initialization is crucial for GAN stability. We use a dedicated “`initialize_weights`” function that sets the weights for all convolutional and batch normalization layers according to the DCGAN paper. All weights are initialized from a normal distribution with a mean of 0.0 and a standard deviation of 0.02.

The DCGAN Training Loop

The core of the project is the “`train_dcgan`” function, where the Generator and Discriminator are trained in an adversarial game. The training for each batch of data involves two main steps:

Update the Discriminator Network

The Discriminator's goal is to get better at telling real images from fake ones. We train it on both:

- **Training on Real Images:** We feed a batch of real images from the CIFAR-10 dataset to the Discriminator. The loss is calculated by comparing the Discriminator's output to the “real” label.
- **Label Smoothing:** To prevent the Discriminator from becoming too confident and to stabilize training, we introduce label smoothing. Instead of using a hard label of 1.0 for real images, we use a slightly smaller value (e.g., 0.9). This small change acts as a “regularizer”, making the training process more robust.
- **Training on Fake Images:** The Generator creates a batch of fake images. These are then fed to the Discriminator. It is crucial here to use `.detach()` on the fake images; this ensures that gradients from this step do not flow back and update the Generator. The loss is calculated by comparing the Discriminator's output to the “fake” label of 0.0.

The total Discriminator loss (`errD`) is the sum of the loss on the real batch and the loss on the fake batch. The Discriminator's weights are then updated to minimize this combined loss.

Update the Generator Network

The Generator's goal is to fool the Discriminator. To do this, we perform the following:

- We pass the fake images (that were generated in the previous step) through the updated Discriminator.
- The Generator's loss (err_G) is calculated by comparing the Discriminator's predictions on these fake images to the "real" label (1.0).
- By minimizing this loss, the Generator learns to produce images that the Discriminator is more likely to classify as "real."

Throughout this process, we keep a "fixed_noise" vector. At regular intervals, we use this same vector to generate a batch of sample images. This allows us to visually monitor the Generator's progress over time and see how the quality of the output evolves.

Evaluating GAN Performance: FID and IS

Evaluating GANs is notoriously difficult because a low loss value doesn't always mean high-quality images. Therefore, we rely on more sophisticated metrics to objectively measure performance. Our `evaluate_model` function is designed to automate this process.

First, we prepare a consistent set of ground-truth images by saving 100 real images from the CIFAR-10 dataset into a directory. This folder serves as our "real" distribution for all subsequent comparisons.

Our evaluation function then calculates two standard metrics:

- **Fréchet Inception Distance (FID):** This metric measures the similarity between the distribution of generated images and the distribution of real images. It uses the InceptionV3 model to extract features from both sets of images and calculates the distance between these feature vectors. A lower FID score is better, indicating that the generated images are statistically similar to real ones in terms of features.
- **Inception Score (IS):** This score measures two things: the quality of the generated images (i.e., do they contain clear, identifiable objects?) and their diversity (i.e., is the generator producing a wide variety of images?). A higher IS is better.

To get these scores, the "evaluate_model" function first instructs the trained Generator to produce and save a large number of images. Then, it uses the "torch_fidelity" library to compute the FID and IS by comparing the folder of generated images against our folder of real images.

Q3: Hyperparameter Sensitivity Analysis

While the DCGAN architecture provides a strong foundation, its performance is highly dependent on various hyperparameter choices. In this section, we conduct a systematic sensitivity analysis to understand how different parameters affect the model's ability to generate quality images.

To do this, we first establish a baseline configuration that serves as our control group. We then create a series of experiments, where each experiment modifies just one parameter from the baseline. Performance for each experiment is measured using the FID and IS metrics we established earlier.

Our baseline configuration is as follows:

- **Noise Vector Size (z_dim):** 100
- **Batch Normalization:** Enabled in both the Generator and the Discriminator.
- **Activation Functions:** ReLU in the Generator's hidden layers and LeakyReLU in the Discriminator's hidden layers.
- **Label Smoothing:** Disabled (factor of 0.0).

Starting from this baseline, we run the following experiments:

1. Input Noise Vector Size (z_dim)

The dimensionality of the latent space can influence the complexity and variety of the generated images. We test two alternative sizes: a smaller latent space of 64 and a larger one of 128. The goal is to see if changing the capacity of the input noise affects the generator's expressive power.

2. Batch Normalization

Batch Normalization is known to be critical for stabilizing GAN training. To test its impact, we run three distinct experiments:

- Removing Batch Normalization from the Generator only.
- Removing Batch Normalization from the Discriminator only.
- Removing Batch Normalization from both networks entirely.

These tests will help quantify how essential BN is for each component of the DCGAN architecture.

3. Label Smoothing

As discussed previously, label smoothing is a regularization technique designed to prevent the discriminator from becoming overconfident. We experiment with two non-zero smoothing factors for the "real" labels: 0.1 and 0.2. This will show whether a "softer" target for the discriminator leads to better generator performance.

4. Activation Functions

The choice of activation function can significantly impact gradient flow and model dynamics. We test alternatives to our baseline's ReLU (Generator) and LeakyReLU (Discriminator):

- Generator Activations: We test LeakyReLU and Tanh in the generator's hidden layers.
- Discriminator Activations: We test ReLU and Tanh in the discriminator's hidden layers.

For each of these configurations, a new DCGAN is initialized, trained for a fixed number of epochs, and then evaluated. The resulting FID and IS scores, along with the configuration parameters, are collected into a final results table to allow for direct comparison.

Results (Numerical) - 5 epochs

--- ALL EXPERIMENT RUNS COMPLETE ---						
	exp_name	z_dim	g_use_bn	d_use_bn	g_activation	d_activation \
0	baseline	100	True	True	relu	leaky_relu
1	z_dim_64	64	True	True	relu	leaky_relu
2	z_dim_128	128	True	True	relu	leaky_relu
3	no_bn_generator	100	False	True	relu	leaky_relu
4	no_bn_discriminator	100	True	False	relu	leaky_relu
5	no_bn_both	100	False	False	relu	leaky_relu
6	label_smooth_0.1	100	True	True	relu	leaky_relu
7	label_smooth_0.2	100	True	True	relu	leaky_relu
8	g_leaky_relu	100	True	True	leaky_relu	leaky_relu
9	g_tanh_hidden	100	True	True	tanh	leaky_relu
10	d_relu	100	True	True	relu	relu
11	d_tanh_hidden	100	True	True	relu	tanh

	label_smooth_factor	lr	beta1	inception_score_mean \
0	0.0	0.0002	0.5	3.063303
1	0.0	0.0002	0.5	3.469573
2	0.0	0.0002	0.5	2.813446
3	0.0	0.0002	0.5	2.648810
4	0.0	0.0002	0.5	1.840050
5	0.0	0.0002	0.5	1.599227
6	0.1	0.0002	0.5	2.772974
7	0.2	0.0002	0.5	3.176057
8	0.0	0.0002	0.5	3.288587
9	0.0	0.0002	0.5	3.011155
10	0.0	0.0002	0.5	3.475556
11	0.0	0.0002	0.5	2.012107

	inception_score_std	frechet_inception_distance	G_loss_final \
0	0.087206	247.004361	2.922569
1	0.234968	234.314585	3.546777
2	0.148763	246.498762	2.810852
3	0.129469	289.816415	5.844398
4	0.064145	310.496396	1.577210
5	0.025645	337.559903	2.192067
6	0.123184	243.198494	2.983086
7	0.128545	237.634418	2.745041
8	0.176470	226.619787	2.748911
9	0.131156	257.889389	3.023351
10	0.264741	236.374997	1.836027
11	0.094244	282.253298	2.163977

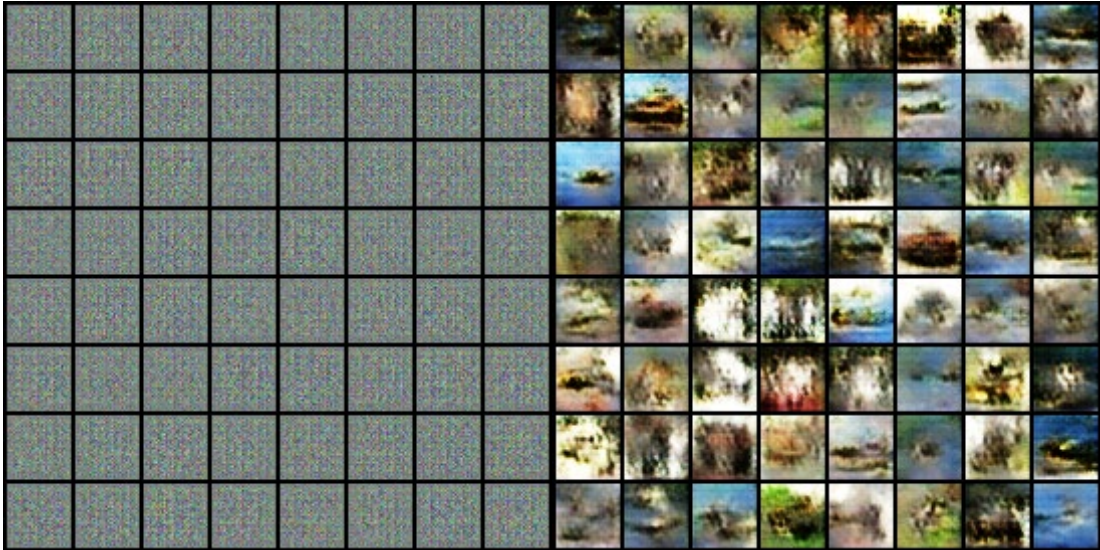
	D_loss_final
0	0.500133
1	0.840272
2	0.468159
3	0.147615
4	1.323474
5	0.878727
6	0.635115
7	0.893334
8	0.405551
9	0.331679
10	0.437155
11	0.931796

After running all 12 experiments, the results table provides a clear picture of which hyperparameters have the most impact.

- **Best Overall Performance:** The experiment named `g_leaky_relu` (row 8) achieved the best FID score of 226.62, while also maintaining a very high Inception Score of 3.29. This combination indicates it produced the most realistic and diverse images among all unconditional DCGANs tested. Using Leaky ReLU in the generator appears to be a significant improvement over the standard ReLU.
- **Impact of Batch Normalization:** The results confirm that Batch Normalization (BN) is absolutely critical for stable training. Removing BN from either the generator or discriminator significantly worsened the FID score. Removing it from both networks (`no_bn_both`, row 5) was catastrophic, resulting in the worst FID (337.56) and the lowest IS (1.60). This shows a complete failure to learn.
- **Impact of Label Smoothing:** Applying label smoothing (rows 6 and 7) resulted in improved FID scores compared to the baseline (243.20 and 237.63 vs. 247.00). This suggests that the regularization technique successfully prevents the discriminator from becoming overconfident and helps the generator learn more effectively.
- **Impact of `z_dim`:** Changing the noise vector size did not show a clear trend. A smaller dimension of 64 (`z_dim_64`, row 1) performed quite well, while a larger dimension of 128 (`z_dim_128`, row 2) performed slightly worse than the baseline.

Results (Visual) - 5 epochs

1. Baseline (start – finish)



2. Z_dim: 64 (start – finish)



3. Z_dim: 128 (start – finish)



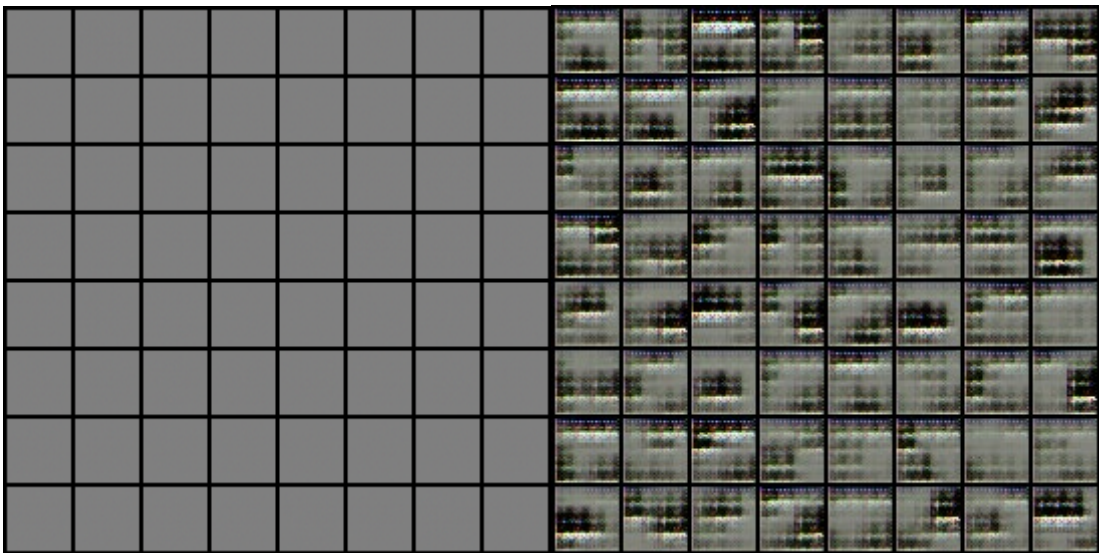
4. No batch_normalization Generator (start – finish)



5. No batch_normalization Discriminator (start – finish)



6. No batch_normalization Both (start – finish)



7. Label Smoothing: 0.1 (start – finish)



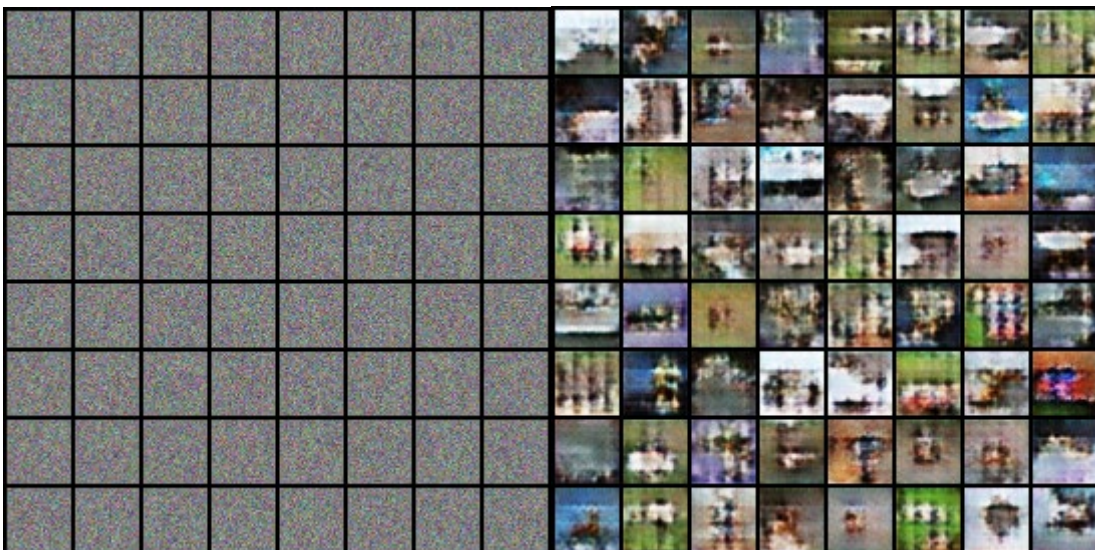
8. Label Smoothing: 0.2 (start – finish)



9. Generator leaky ReLU (start – finish)



10. Generator Tanh hidden (start – finish)



11. Discriminator ReLU (start – finish)



12. Discriminator Tanh hidden (start – finish)



The generated images visually confirm the findings from the numerical results.

- The visual output for the `no_bn_both` experiment (Page 13, Image 6) clearly shows mode collapse. The generator fails to produce anything other than repetitive, noisy patterns, which explains its terrible FID and IS scores.
- In contrast, the images from the best-performing models, such as `g_leaky_relu` (Page 14, Image 9) and the label-smoothed models (Page 13, Image 7 and Page 14, Image 8), appear noticeably more diverse and contain more discernible shapes and colors than the baseline model.
- Across all successful experiments, the images progress from pure random noise at the beginning of training to blurry but varied and colorful outputs by the end of the 5 epochs, demonstrating that the models are learning to capture the features of the CIFAR-10 dataset.

Q4: Conditional GAN (CGAN)

A major limitation of the standard DCGAN is that we have no control over the images it produces. The model learns the underlying distribution of the entire dataset, but we cannot ask it to generate an image of a specific class. The Conditional GAN (CGAN) solves this problem by providing a mechanism to direct the generation process.

The core idea of a CGAN is to "condition" both the Generator and the Discriminator on additional information—in our case, the class label of the image.

Architectural Modifications

To incorporate this conditional information, we modify our base DCGAN architecture:

Generator (Generator_CGAN):

- The Generator now accepts a class label along with the random noise vector as input.
- It uses a new `nn.Embedding` layer, which converts the integer class label (e.g., 3 for 'cat') into a dense feature vector.
- This class embedding is then directly concatenated with the initial noise vector. This combined vector now contains both random variation (from the noise) and class-specific direction (from the embedding), guiding the generator to create an image of the desired category.

Discriminator (Discriminator_CGAN):

- The Discriminator's task is now more complex: it must determine if an image is real and if it matches its given class label.

- Like the Generator, it is also equipped with an “nn.Embedding” layer to process the class label.
- The label embedding is reshaped into a 2D feature map and concatenated with the input image as an extra channel. This allows the discriminator's convolutional layers to process both the image's visual features and its conditional information simultaneously.

Training and Conditional Generation

The training loop is updated to leverage these new architectures. During each step, the real class labels are fed to both the Generator (to guide fake image creation) and the Discriminator (to help it judge both real and fake images against their labels).

The true power of the CGAN is demonstrated in the “generate_for_class” function. After training, we can use this function to request images of a specific class. By providing the trained generator with noise and a fixed class label (e.g., the label for 'cat'), it produces images exclusively from that category, giving us direct control over the output that was not possible with the previous models

Results (All Classes)

```
Epoch [1/5]      Loss D: 0.6848      Loss G: 2.9232
Epoch [2/5]      Loss D: 0.6248      Loss G: 2.3583
Epoch [3/5]      Loss D: 0.7347      Loss G: 2.2922
Epoch [4/5]      Loss D: 0.7059      Loss G: 2.2849
Epoch [5/5]      Loss D: 0.5177      Loss G: 2.3999
Training Complete.

Evaluating cgan_final_run: Generating 1000 images...
Generating for cgan_final_run: 100%|██████████| 1000/1000 [00:04<00:00, 224.61it/s]
Calculating IS and FID for cgan_final_run...

--- Results for cgan_final_run ---
Inception Score (IS): 4.5496 ± 0.6991
Frechet Inception Distance (FID): 274.9393
-----
```



The CGAN achieved an Inception Score of 4.55 and an FID of 274.94. The “IS” is significantly higher than any of the DCGAN models, indicating that the generated images are of higher quality and diversity.

Results (Specific Classes)

Generated Images for Class: cat



Generated Images for Class: car



Generated Images for Class: bird



Conditional Generation: The model successfully generates images that are clearly recognizable as the requested class ('cat', 'car', 'bird'). While not photorealistic, the generated images consistently contain the correct general shapes, colors, and features for their target class. This proves that we have gained direct control over the generator's output, which was the primary goal of this architecture

Q5: Wasserstein GAN with Gradient Penalty (WGAN-GP)

While DCGANs are powerful, they are often difficult to train. They can suffer from problems like mode collapse (where the generator produces only a limited variety of images) and unstable, oscillating losses that don't correlate well with image quality. To address these issues, we implement a more advanced architecture: the Wasserstein GAN with Gradient Penalty (WGAN-GP).

The key innovation of WGAN is to change the loss function. Instead of training a discriminator to perform binary classification (real vs. fake), it trains a "Critic" to approximate the Earth-Mover (or Wasserstein) distance between the real and generated data distributions. This distance provides a much smoother and more meaningful loss signal, which greatly stabilizes training.

Architectural and Methodological Changes

Implementing WGAN-GP requires several significant changes from our DCGAN baseline:

- **The Critic:** We replace the Discriminator with a Critic_WGAN model. This new model has two critical architectural differences:
 1. The final `nn.Sigmoid` activation function is removed. The Critic's job is not to output a probability, but a raw, unbounded score.
 2. Batch Normalization layers are removed. This is a requirement for the gradient penalty to work correctly.
- **Gradient Penalty (GP):** To make the Wasserstein distance loss work, the Critic must satisfy a mathematical property called the "1-Lipschitz constraint." WGAN-GP enforces this by adding a gradient penalty to the Critic's loss. Our `compute_gradient_penalty` function does this by creating interpolated images between real and fake samples and calculating the gradient of the Critic's output with respect to these interpolations. The loss then penalizes any gradients whose norm deviates from 1. This is a more effective and stable approach than the "weight clipping" method used in the original WGAN paper.
- **The WGAN-GP Loss Function:** The binary cross-entropy loss is replaced entirely:
 - **Critic Loss:** The Critic tries to maximize the difference between the scores of real and fake images. Its loss is calculated as: $E[\text{critic}(\text{fake})] - E[\text{critic}(\text{real})] + \text{gradient_penalty}$.
 - **Generator Loss:** The Generator's goal is to produce images that receive a high score from the Critic. Therefore, its loss is simply: $-E[\text{critic}(\text{fake})]$.

Training Process

The training dynamic for WGAN-GP is also different. To ensure the Critic provides a reliable gradient to the Generator, we update it more frequently. In our implementation, the Critic is trained for 5 iterations for every single training iteration of the Generator. This asymmetric training schedule is crucial for the stability of the WGAN-GP framework. Finally, the trained generator is evaluated using the same FID and IS metrics for comparison against the other models.

Results (epoch 1 – 5)

```
Epoch [1/5]      Loss C: -5.3183      Loss G: -32.7375
Epoch [2/5]      Loss C: -5.1856      Loss G: -15.2999
Epoch [3/5]      Loss C: -3.8844      Loss G: -20.0674
Epoch [4/5]      Loss C: -3.1751      Loss G: -20.4776
Epoch [5/5]      Loss C: -2.6772      Loss G: -17.4324
Training Complete.

--- Evaluating WGAN-GP Model ---
Evaluating wgan_gp_final_run: Generating 1000 images...
Generating for wgan_gp_final_run: 100%|██████████| 1000/1000 [00:05<00:00, 167.00it/s]
Calculating IS and FID for wgan_gp_final_run...
Results for wgan_gp_final_run:
  Inception Score (IS): mean=4.5496, std=0.6991
  Frechet Inception Distance (FID): 245.6219
```

The WGAN-GP achieved an Inception Score of 4.55 and an FID of 245.62.

The IS of 4.55 is top-tier, matching the high score of the CGAN and indicating very high-quality image generation.

The FID of 245.62 is very strong, outperforming the baseline DCGAN and the CGAN, and landing very close to the best-performing DCGAN from the sensitivity tests.



The generated images show good diversity and quality, supporting the excellent numerical scores. Overall, WGAN-GP proves to be a superior architecture, providing a powerful combination of high image quality (high IS), strong realism (low FID), and reliably stable training.

Q6: Distribution Visualization with t-SNE

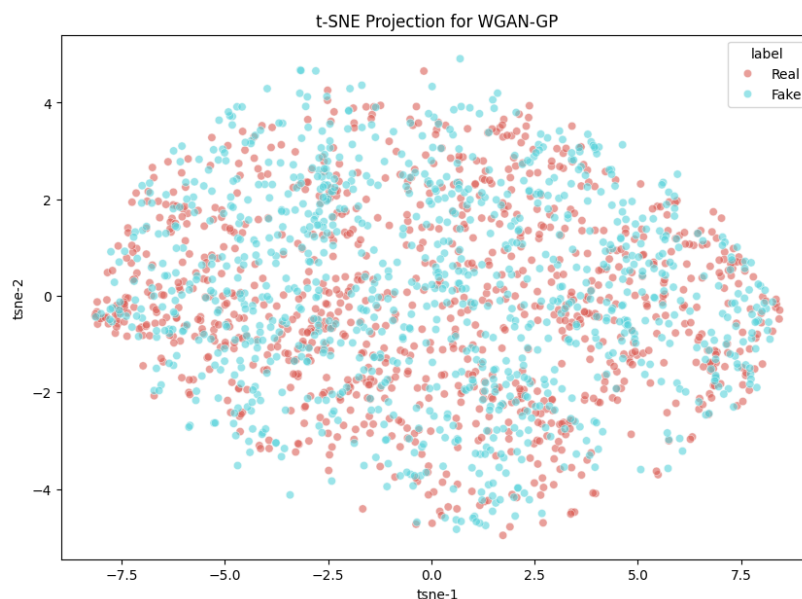
While quantitative metrics like FID and IS are essential for evaluation, a visual, qualitative assessment can provide deeper insight into how well our generator is learning. The ultimate goal is for the distribution of generated ("fake") data to match the distribution of real data. To visualize this, we use a powerful dimensionality reduction technique called t-SNE.

The `visualize_tsne` function is designed to create a 2D scatter plot that shows us how the features of real and fake images relate to each other.

Methodology

1. **Feature Extraction:** Instead of comparing raw pixels, we compare high-level features. We use our trained WGAN-GP Critic as a feature extractor by taking all of its convolutional layers and removing the final output layer. These intermediate layers have learned to identify the most salient features for distinguishing between real and fake images.
2. **Data Generation:** We pass a large sample of both real images (from the dataset) and fake images (from our trained generator) through this feature extractor to get a high-dimensional feature vector for each image.
3. **t-SNE Projection:** We feed the combined set of real and fake feature vectors into the t-SNE algorithm. t-SNE maps these high-dimensional vectors into a 2D space while attempting to preserve the local neighborhood structure of the original data.
4. **Plotting:** The resulting 2D points are plotted on a scatter plot, with points colored based on whether they came from a 'Real' or 'Fake' image.

Interpreting the Results



The t-SNE plot gives a clear, qualitative confirmation of the WGAN-GP's success. The plot visualizes the high-dimensional features of both real and fake images in 2D space.

We can see that the 'Real' (pink) and 'Fake' (light blue) data points are heavily overlapped and well-mixed. There is no clean separation between the two clusters.

This is an excellent result. It visually demonstrates that the WGAN-GP generator is producing images with features so similar to the real images that they are difficult to distinguish, even in this reduced feature space. This confirms that the generator has successfully learned the underlying distribution of the real data.

Q7: Analysis of GANs

GANs vs. Other Generative Models (like VAEs)

The primary advantage of Generative Adversarial Networks (GANs) is their ability to produce sharp, realistic images, which often surpass the blurrier results from Variational Autoencoders (VAEs). However, this quality comes at the cost of training stability; the adversarial process is notoriously difficult to balance, whereas VAEs train more reliably. GANs can also suffer from mode collapse, failing to capture the full diversity of the data, a problem less common in VAEs.

Core Training Challenges

Training GANs is difficult due to three main issues:

- **Instability:** The training is a delicate balancing act. If the discriminator becomes too powerful, the generator gets no useful feedback and fails to learn.
- **Mode Collapse:** The generator may learn to produce only a limited variety of samples that can easily fool the discriminator, failing to capture the full diversity of the dataset.
- **Difficult Convergence:** GAN losses naturally oscillate, making it hard to judge when a model has finished training. This is why external metrics like FID and IS are essential for evaluation.

Solutions for Stable Training

Several effective strategies exist to mitigate these challenges, many of which were demonstrated in this project:

- **Improved Architectures:** Using the DCGAN architecture with Batch Normalization and LeakyReLU activations provides a much more stable foundation than earlier GANs.

- **Advanced Loss Functions:** The WGAN-GP architecture significantly improves stability by replacing the standard loss with the Wasserstein distance and a gradient penalty, which provides a more reliable learning signal.
- **Regularization:** Simple techniques like label smoothing prevent the discriminator from becoming overconfident, which helps the generator learn more effectively.
- **Conditional Information:** Providing class labels to the network, as done in the CGAN, can also stabilize training by breaking the problem down into smaller, more manageable tasks.